

you want to classify. For example, if you're classifying dog breeds, collect images of different breeds.

- 2. Label Images:** Create labels for each image that correspond to its category.
- 3. Split Dataset:** Divide your dataset into training (80%) and validation (20%) sets.

Step 3: Set Up Your Development Environment

- 1. Install Libraries:** Ensure you have the necessary libraries installed. Use the following commands to install TensorFlow or PyTorch:

```
pip install tensorflow
pip install torch torchvision
```

- 2. Prepare Data Loaders:** Use data loaders to handle the loading and preprocessing of images. Below is an example using PyTorch:

```
import torch
from torchvision import datasets, transforms

# Define transforms
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])

# Load datasets
train_dataset = datasets.ImageFolder(
    root='path_to_train_data',
    transform=transform)
val_dataset = datasets.ImageFolder(
    root='path_to_val_data',
    transform=transform)

# Create data loaders
train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size=32,
    shuffle=True)
val_loader = torch.utils.data.DataLoader(
    val_dataset, batch_size=32,
    shuffle=False)
```

Step 4: Fine-Tune the Model

- 1. Load Pre-Trained Model:** Load the pre-trained ResNet model and modify the final layer to match the number of categories in your dataset.

```
from torchvision import models
# Load pre-trained ResNet model
model = models.resnet18(pretrained=True)
# Modify the final layer
num_fts = model.fc.in_features
model.fc = torch.nn.Linear(num_fts, num_classes)
```

- 2. Define Loss Function and Optimizer:** Choose an appropriate loss function and optimizer.

```
import torch.optim as optim
criterion = torch.nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(),
    lr=0.001, momentum=0.9)
```

- 3. Train the Model:** Train the model using the training dataset and validate its performance using the validation dataset.

```
num_epochs = 10
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    for inputs, labels in train_loader:
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {running_loss/len(train_loader):.4f}')
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for inputs, labels in val_loader:
            outputs = model(inputs)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    print(f'Validation Accuracy: {100 * correct / total:.2f}%')
```

- 4. Save the Fine-Tuned Model:** Save the trained model for future use.

```
torch.save(model.state_dict(), 'fine_tuned_resnet.pth')
```

Implementing Image Recognition

Fine-tuning an image recognition model can significantly improve its accuracy for specific tasks. Here's how you can implement and use your fine-tuned model for image recognition.

Step 1: Load the Fine-Tuned Model

- 1. Load Model Weights:** Load the weights of your fine-tuned model.

```
model.load_state_dict(torch.load('fine_tuned_resnet.pth'))
model.eval()
```